**МІНІСТЕРСТВО ОСВІТИ УКРАЇНИ**

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ**
**«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ**

**імені Ігоря Сікорського»**
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
«ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ»

Кафедра Системного проектування

Лабораторна робота №2

з дисципліни “Інтелектуальний аналіз даних”

# на тему: “Розробка методу класифікації даних Naive Bayes”

Виконав:
Студенд 2-го курсу
Групи ДА-11
Сидько Нікіта Сергійович
Варіант 22

Київ – 2023

Мета роботи: Здобути навички програмної розробки методу класифікації.

Завдання: Розробити програму для класифікації даних.

## Хід роботи

### 1.Підготовка набору даних

Для виконання умови завдання зображеному на рис.1

Нехай вирішується задача класифікації листів з електронної скриньки на наявність спаму. Деякі електронні листи вже мають відмітки “Спам”, інші вважаються – “Не спам”. Необхідно розробити програму, яка буде аналізувати назви нових листів і повідомляти про можливість спаму з певним відсотком ймовірності.

Рис.1

Створюємо структуру даних, що має поля string (для запису повідомлення ) та bool (для позначення спам/не спам)

```
struct Type_of_message
{
    string message;
    bool isSpam = 0;
};
```

Рис.2

Далі створюємо вектор, для зберігання train даних та заповнюємо його повідомленнями (true – спам повідомлення, false – не спам повідомлення)

```
vector <Type_of_message> first_six_messages;
Type_of_message A, B, C, D, E, F;
A.isSpam = true;
A.message = "Hello special sales on boots";
B.isSpam = true;
B.message = "Hi discout on shoes";
C.isSpam = false;
C.message = "Hey rebate on snikers";
D.isSpam = false;
D.message = "Hello special discout on boots";
E.isSpam = true;
E.message = "Hi rebate on shoes";
F.isSpam = false;
F.message = "Hey some sales for you";

first_six_messages.push_back(A);
first_six_messages.push_back(B);
first_six_messages.push_back(C);
first_six_messages.push_back(D);
first_six_messages.push_back(E);
first_six_messages.push_back(F);
```

Рис.3

## 2. Опис роботи коду

Спочатку створюємо словник даних, в моєму випадку це буде unordered_map words, що буде мати пару ключ – слово, значення – кількість повторювань цього слова у повідомленнях (рис.4). Далі зробимо функцію WordCounter() (рис.5), яка буде зчитувати з масиву повідомлення, розбиваючи його на слова, та підраховувати кількість повторень слів. Також ця функція буде заповнювати unordered_map spam_counter.

```
unordered_map <string, int> words; // зберігання всіх слів, і їх кількості
unordered_map <string, int> spam_counter; // зберігання кількості СПАМ слів
unordered_map <string, double> spam_percentage; // зберігання слово - відсоток спаму
unordered_map <string, double> norm_spam_percentage; // зберігання слово - нормальний відсоток спаму
```

Рис.4

```
// функція для підрахунку слів
void WordCounter(vector <Type_of_message> &messages , unordered_map <string, int>&words, unordered_map <string, int>&spam_counter)
{
    int iterator = 0;
    for (auto& i : messages)
    {
        istringstream iss(messages[iterator].message);
        string word;
        while (iss >> word)
        {
            words[word]++;

            if (messages[iterator].isSpam == true)
            {
                spam_counter[word]++;
            }
            if (messages[iterator].isSpam == false)
            {
                spam_counter[word] + 0;
            }
        }
        iterator++;
    }
}
```

Рис.5

На рисунку 6, зображено те, що виконує функція WordCounter.

```
Консоль отладки Microsoft Visual Studio
Hello special sales on boots  (SPAM)
Hi discout on shoes  (SPAM)
Hey rebate on snikers  (NOT SPAM)
Hello special discout on boots  (NOT SPAM)
Hi rebate on shoes  (SPAM)
Hey some sales for you  (NOT SPAM)

Word: shoes      Amount: 2
Word: Hello      Amount: 2
Word: for        Amount: 1
Word: on         Amount: 5
Word: discout    Amount: 2
Word: snikers    Amount: 1
Word: special    Amount: 2
Word: sales      Amount: 2
Word: Hi         Amount: 2
Word: boots      Amount: 2
Word: Hey        Amount: 2
Word: rebate     Amount: 2
Word: some       Amount: 1
Word: you        Amount: 1

Word: shoes      Amount: 2
Word: Hello      Amount: 1
Word: for        Amount: 0
Word: on         Amount: 3
Word: discout    Amount: 1
Word: snikers    Amount: 0
Word: special    Amount: 1
Word: sales      Amount: 1
Word: Hi         Amount: 2
Word: boots      Amount: 1
Word: Hey        Amount: 0
Word: rebate     Amount: 1
Word: some       Amount: 0
Word: you        Amount: 0
```

Рис.6

Використовуючи кількість повторень слів можна розрахувати ймовірність їх відношення до певного класу. Тому використовуючі дані про слова з unordered_map words та unordered_map spam_counter, зробимо обрахунки ймовірності використовуючи функцію SpamProbability() зображену на рис. 7;

```
void SpamProbability(unordered_map <string, int>& words, unordered_map <string, int>& spam_counter, unordered_map <string, double>& unorder_map)
{
    for (auto& pair : words)
    {
        double spam = double(spam_counter[pair.first] / double(words[pair.first]));
        unorder_map.emplace(pair.first, spam);
    }
}
```

Рис.7

На рис.8 зображено те, що виконує функція SpamProbability()

```
Word: Hello      Spam : 0.5      Not spam : 0.5
Word: shoes      Spam : 1        Not spam : 0
Word: snikers    Spam : 0        Not spam : 1
Word: special    Spam : 0.5      Not spam : 0.5
Word: for        Spam : 0        Not spam : 1
Word: on         Spam : 0.6      Not spam : 0.4
Word: discout    Spam : 0.5      Not spam : 0.5
Word: sales      Spam : 0.5      Not spam : 0.5
Word: Hi         Spam : 1        Not spam : 0
Word: boots      Spam : 0.5      Not spam : 0.5
Word: Hey        Spam : 0        Not spam : 1
Word: rebate     Spam : 0.5      Not spam : 0.5
Word: some       Spam : 0        Not spam : 1
Word: you        Spam : 0        Not spam : 1
```

Рис.8

Якщо ймовірність хоча б одного слова у назві рівна нулю загальна ймовірність також буде рівна нулю. Щоб уникнути цього, усі ймовірності у словнику можна нормувати за наступною формулою, що зображена на рис.9

$$p_{норм} = \frac{n_{слово} \cdot p_{ненорм} + p_{класу}}{n_{слово} + 1}$$

Рис.9

Напишемо функцію NormSpamProbability() яка буде рахувати нормовану ймовірність

```
void NormSpamProbability(unordered_map <string, int> &words, unordered_map <string, double> &spam_percentage, unordered_map <string, double> &norm_spam_percentage)
{
    for (auto& pair : words)
    {
        double p_norm_spam = (double(pair.second) * double(spam_percentage[pair.first]) + 0.5) / (double(pair.second) + 1);
        p_norm_spam = round(p_norm_spam * 100) / 100;
        norm_spam_percentage.emplace(pair.first, p_norm_spam);
    }
}
```

Рис.10

На рис.11 зображено виконання роботи функції NormSpamProbability()

```
Word: Hello      Spam : 0.5      Not spam : 0.5
Word: shoes      Spam : 0.83     Not spam : 0.17
Word: snikers    Spam : 0.25     Not spam : 0.75
Word: special    Spam : 0.5      Not spam : 0.5
Word: for        Spam : 0.25     Not spam : 0.75
Word: on         Spam : 0.58     Not spam : 0.42
Word: discout    Spam : 0.5      Not spam : 0.5
Word: sales      Spam : 0.5      Not spam : 0.5
Word: Hi         Spam : 0.83     Not spam : 0.17
Word: boots      Spam : 0.5      Not spam : 0.5
Word: Hey        Spam : 0.17     Not spam : 0.83
Word: rebate     Spam : 0.5      Not spam : 0.5
Word: some       Spam : 0.25     Not spam : 0.75
Word: you        Spam : 0.25     Not spam : 0.75
```

Рис.11

Для того, щоб класифікувати назву листа необхідно мати словник слів та їх ймовірності. Якщо класів слів два, як у нашому прикладі, необхідно для кожного класу з ймовірністю 0.5 розрахувати загальну ймовірність приналежних слів за наступною формулою зображеною на рис.12

$$p_{повідом} = p_{клас} \cdot \prod_{i=1}^{n} p_{слово}[i],$$

Рис.12

Для цього напишемо дві функції Spam_Percentage_Calculate() рис.13 і Not_spam_Percentage_Calculate() рис.14 . Ці дві функції будуть підраховувати ймовірність принадлежності письма до спам і не спам нових повідомлень.

```
vector<double> Spam_Percentage_Calculate(vector<Type_of_message>& new_messages, unordered_map<string, double>& norm_spam_percentage)
{
    std::vector<double> spam_percentages;
    int iterator = 0;
    for (auto& message : new_messages)
    {
        double percent = 1.0;
        istringstream iss(new_messages[iterator].message);;
        string word;
        while (iss >> word)
        {
            auto it = norm_spam_percentage.find(word);
            if (it != norm_spam_percentage.end())
            {
                percent *= it->second;
            }
            else
            {
                percent *= 0.5; // новое слово
            }
        }
        spam_percentages.push_back(percent);
        iterator++;
    }
    for (int i = 0; i < spam_percentages.size(); i++)
    {
        spam_percentages[i] *= 0.5;
    }
    return spam_percentages;
}
```

Рис.13

```cpp
vector<double> Not_spam_Percentage_Calculate( vector<Type_of_message>& new_messages, unordered_map<string, double>& norm_spam_percentage)
{
    std::vector<double> not_spam_percentages;
    int iterator = 0;
    for (auto& message : new_messages)
    {
        double percent = 1.0;
        std::istringstream iss(new_messages[iterator].message); //
        std::string word;
        while (iss >> word)
        {
            auto it = norm_spam_percentage.find(word);
            if (it != norm_spam_percentage.end())
            {
                percent *= 1-(it->second);
            }
            else
            {
                percent *= 0.5; // новое слово
            }
        }
        not_spam_percentages.push_back(percent);
        iterator++;
    }
    for (int i = 0; i < not_spam_percentages.size(); i++)
    {
        not_spam_percentages[i] *= 0.5;
    }
    return not_spam_percentages;
}
```

Рис.14

Тестування коду :

Запишемо нові повідомлення (рис.15)

```cpp
vector <Type_of_message> new_messages;
Type_of_message message1, message2, message3, message4;
message1.isSpam = true;
message1.message = "Hello rebate shoes";
message2.isSpam = false;
message2.message = "Hi sales on shoes";
message3.isSpam = false;
message3.message = "Hey discount on snikers";
message4.isSpam = true;
message4.message = "Only today special sales on boots";

new_messages.push_back(message1);
new_messages.push_back(message2);
new_messages.push_back(message3);
new_messages.push_back(message4);
```

Рис.15

Використаємо наші функції для підрахунку % належності повідомлення до спаму/не спаму

```cpp
vector<double> spam_percentages = Spam_Percentage_Calculate(new_messages, norm_spam_percentage);
vector<double> not_spam_percentages = Not_spam_Percentage_Calculate(new_messages, norm_spam_percentage);
vector<string>meaning;
for (int i = 0; i < spam_percentages.size(); i++)
{
    if (0.5 * spam_percentages[i] > 0.5 * not_spam_percentages[i])
    {
        meaning.push_back(" SPAM");
        cout << "Message: " << new_messages[i].message <<" [isSpam: "<<new_messages[i].isSpam <<"]" << "\t Spam " << spam_percentages[i] << "\t" << " Not spam " << not_spam_percentages[i] << meaning[i] << endl;
    }
    if (0.5 * spam_percentages[i] < 0.5 * not_spam_percentages[i])
    {
        meaning.push_back(" NOT SPAM");
        cout << "Message: " << new_messages[i].message <<" [isSpam: " << new_messages[i].isSpam << "]" <<  "\t  Spam " << spam_percentages[i] << "\t" << " Not spam " << not_spam_percentages[i] << meaning[i] << endl;
    }
    cout << endl;
}
```

Рис.16

Далі запишемо цикл, для підрахунку коректності роботи програми

```
double correctness_of_work = 0;
double a = 0;
double b = 0;
for (int i = 0; i < meaning.size(); i++)
{
    if (new_messages[i].isSpam == true && meaning[i] == " SPAM")
    {
        a++;
        b++;
    }
    if (new_messages[i].isSpam == false && meaning[i] == " NOT SPAM")
    {
        a++;
        b++;
    }
    if (new_messages[i].isSpam == true && meaning[i] != " SPAM")
    {
        b++;
    }
    if (new_messages[i].isSpam == false && meaning[i] != " NOT SPAM")
    {
        b++;
    }

}
correctness_of_work = a / b;
cout <<"Correctness of work : "<< correctness_of_work <<" % " << endl;
```

Рис.17

Результат роботи коду:

```
Message: Hello rebate shoes [isSpam: 1]  Spam 0.10375    Not spam 0.02125 SPAM

Message: Hi sales on shoes [isSpam: 0]   Spam 0.0998905  Not spam 0.0030345 SPAM

Message: Hey discount on snikers [isSpam: 0]      Spam 0.0061625         Not spam 0.0653625 NOT SPAM

Message: Only today special sales on boots [isSpam: 1]   Spam 0.0090625  Not spam 0.0065625 SPAM

Correctness of work : 0.75 %
```

Рис.18

Побудуємо криву залежності (рис.19) якості класифікації відповідно до кількості екземплярів

| X | Y |
|---|---|
| 1 | 100 |
| 2 | 50 |
| 3 | 66,6 |
| 4 | 75 |

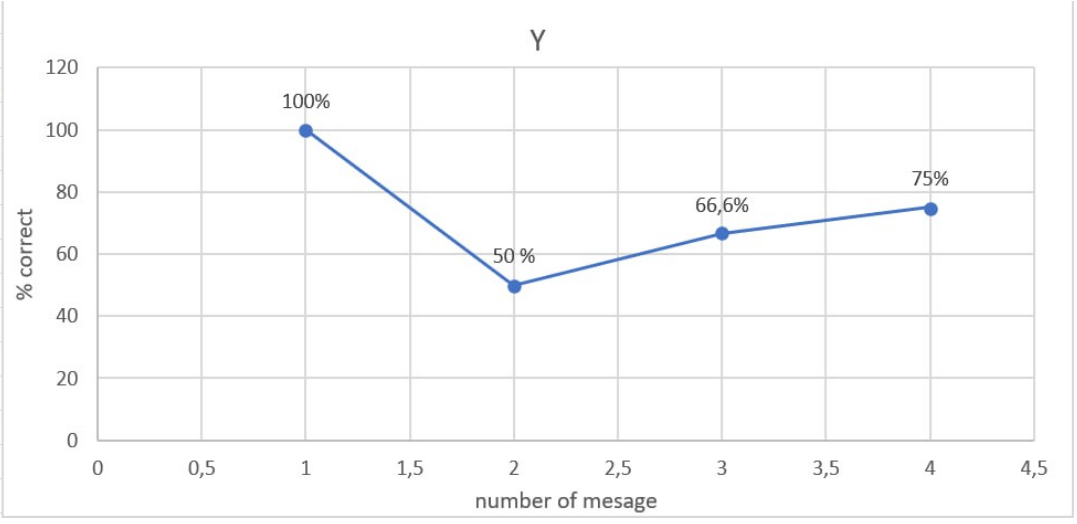



Рис.19

**Висновок.**

У ході виконання лабораторної роботи №2 було розроблено програму для підрахунку відсотка належності повідомлення до класу спам або не спам і подальшого їх класифікування за допомогу метода “ Naive Bayes ”. Програма зчитує дані з нових листів і за допомогою формул розраховує відсоток належності письма до класу спам/не спам після чого робить висновок до якого класу належить повідомлення. Аналіз проводився по словам що зустрічаються у назвах листів. Для тренування було обрано 2/3 частина від усіх повідомлень, а тестування – 1/3. За тестувальним набором даних було отримано 75% коректності роботи програми. Також була побудована крива залежності якості класифікації.

**Лістинг 1**

```
#include <iostream>
#include <string>
#include <vector>
#include <unordered_map>
#include <sstream>

using namespace std;

// тип даних, який буде використовуватися для позначення назви листа + спам/не спам
struct Type_of_message
{
	string message;
	bool isSpam = 0; // true - spam, false - not spam
};

// функція для підрахунку слів
void WordCounter(vector <Type_of_message> &messages , unordered_map <string,
int>&words, unordered_map <string, int>&spam_counter)
{
	int iterator = 0;
	for (auto& i : messages)
	{
		istringstream iss(messages[iterator].message);
		string word;
		while (iss >> word)
		{
			words[word]++;

			if (messages[iterator].isSpam == true)
			{
				spam_counter[word]++;
			}
			if (messages[iterator].isSpam == false)
			{
				spam_counter[word] + 0;
			}
```

```
			}
			iterator++;
		}
}
//функція для показу словника у консолі
void ShowWords( unordered_map <string, int> &unorder_map )
{
		cout << endl;
		for (auto& pair : unorder_map)
		{
			cout << "Word: " << pair.first << "\t " << "Amount: " << pair.second <<
endl;
		}
}
// функція для підрахунку % спаму
void SpamProbability(unordered_map <string, int>& words, unordered_map <string, int>&
spam_counter, unordered_map <string, double>& unorder_map)
{
		for (auto& pair : words)
		{
			double spam = double(spam_counter[pair.first] /
double(words[pair.first]));
			unorder_map.emplace(pair.first, spam);
		}

}
// функція для підрахунку нормального % спаму
void NormSpamProbability(unordered_map <string, int> &words, unordered_map <string,
double> &spam_percentage, unordered_map <string, double> &norm_spam_percentage)
{
		for (auto& pair : words)
		{
			double p_norm_spam = (double(pair.second) *
double(spam_percentage[pair.first]) + 0.5) / (double(pair.second) + 1);
			p_norm_spam = round(p_norm_spam * 100) / 100;
			norm_spam_percentage.emplace(pair.first, p_norm_spam);
		}
}
//функція для показу словника у консолі
void ShowPersentage(unordered_map <string, double>& unorder_map)
{
		cout << endl;
		for (auto& pair : unorder_map)
		{
			cout << "Word: " << pair.first << "\t " << "Spam : " << pair.second <<
"\t Not spam : "<< 1 - pair.second << endl;
		}
}
// функція, що підраховує % належності листа до спаму
vector<double> Spam_Percentage_Calculate(vector<Type_of_message>& new_messages,
unordered_map<string, double>& norm_spam_percentage)
{
		std::vector<double> spam_percentages;
		int iterator = 0;
		for (auto& message : new_messages)
		{
			double percent = 1.0;
			istringstream iss(new_messages[iterator].message); // розділення і-го
елемента масиву повідомлень на слова
			string word;
			while (iss >> word)
			{
				auto it = norm_spam_percentage.find(word);
```

```
                if (it != norm_spam_percentage.end())
                {
                        percent *= it->second;
                }
                else
                {
                        percent *= 0.5; // новое слово
                }
        }
        spam_percentages.push_back(percent);
        iterator++;
    }
    for (int i = 0; i < spam_percentages.size(); i++)
    {
        spam_percentages[i] *= 0.5;
    }
    return spam_percentages;
}
// функція, що підраховує % належності листа до не спаму
vector<double> Not_spam_Percentage_Calculate( vector<Type_of_message>& new_messages,
unordered_map<string, double>& norm_spam_percentage)
{
    std::vector<double> not_spam_percentages;
    int iterator = 0;
    for (auto& message : new_messages)
    {
        double percent = 1.0;
        std::istringstream iss(new_messages[iterator].message); // розділення і-
го елемента масиву повідомлень на слова
        std::string word;
        while (iss >> word)
        {
                auto it = norm_spam_percentage.find(word);
                if (it != norm_spam_percentage.end())
                {
                        percent *= 1-(it->second);
                }
                else
                {
                        percent *= 0.5; // якщо нове слово
                }
        }
        not_spam_percentages.push_back(percent);
        iterator++;
    }
    for (int i = 0; i < not_spam_percentages.size(); i++)
    {
        not_spam_percentages[i] *= 0.5;
    }
    return not_spam_percentages;
}

int main()
{
    vector <Type_of_message> first_six_messages;
    Type_of_message A, B, C, D, E, F;
    A.isSpam = true;
    A.message = "Hello special sales on boots";
    B.isSpam = true;
    B.message = "Hi discout on shoes";
    C.isSpam = false;
    C.message = "Hey rebate on snikers";
    D.isSpam = false;
```

```
	D.message = "Hello special discout on boots";
	E.isSpam = true;
	E.message = "Hi rebate on shoes";
	F.isSpam = false;
	F.message = "Hey some sales for you";

	first_six_messages.push_back(A);
	first_six_messages.push_back(B);
	first_six_messages.push_back(C);
	first_six_messages.push_back(D);
	first_six_messages.push_back(E);
	first_six_messages.push_back(F);

	for (int i = 0; i < first_six_messages.size(); i++) // вивід у консоль
початкових даних
	{
		if (first_six_messages[i].isSpam)
		{
			cout << first_six_messages[i].message << " " << " (SPAM) " <<
endl;
		}
		else
		{
			cout << first_six_messages[i].message << " " << " (NOT SPAM) " <<
endl;
		}
	}

	unordered_map <string, int> words; // зберігання всіх слів, і їх кількості
	unordered_map <string, int> spam_counter; // зберігання кількості СПАМ слів
	unordered_map <string, double> spam_percentage; // зберігання слово - відсоток
спаму
	unordered_map <string, double> norm_spam_percentage; // зберігання слово -
нормальний відсоток спаму

	WordCounter(first_six_messages, words, spam_counter);
	ShowWords(words);
	ShowWords(spam_counter);

	SpamProbability(words, spam_counter, spam_percentage);
	ShowPersentage(spam_percentage);

	NormSpamProbability(words, spam_percentage, norm_spam_percentage);
	ShowPersentage(norm_spam_percentage);

	cout << endl;
	vector <Type_of_message> new_messages;
	Type_of_message message1, message2, message3, message4;
	message1.isSpam = true;
	message1.message = "Hello rebate shoes";
	message2.isSpam = false;
	message2.message = "Hi sales on shoes";
	message3.isSpam = false;
	message3.message = "Hey discount on snikers";
	message4.isSpam = true;
	message4.message = "Only today special sales on boots";

	new_messages.push_back(message1);
	new_messages.push_back(message2);
	new_messages.push_back(message3);
	new_messages.push_back(message4);
```

```
	vector<double> spam_percentages = Spam_Percentage_Calculate(new_messages,
norm_spam_percentage);
	vector<double> not_spam_percentages =
Not_spam_Percentage_Calculate(new_messages, norm_spam_percentage);
	vector<string>meaning;
	for (int i = 0; i < spam_percentages.size(); i++)
	{
		if (0.5 * spam_percentages[i] > 0.5 * not_spam_percentages[i])
		{
			meaning.push_back(" SPAM");
			cout << "Message: " << new_messages[i].message <<" [isSpam:
"<<new_messages[i].isSpam <<"]" << "\t Spam " << spam_percentages[i] << "\t" << " Not
spam " << not_spam_percentages[i] << meaning[i] << endl;
		}
		if (0.5 * spam_percentages[i] < 0.5 * not_spam_percentages[i])
		{
			meaning.push_back(" NOT SPAM");
			cout << "Message: " << new_messages[i].message <<" [isSpam: " <<
new_messages[i].isSpam << "]" <<  "\t  Spam " << spam_percentages[i] << "\t" << " Not
spam " << not_spam_percentages[i] << meaning[i] << endl;
		}
		cout << endl;
	}

	double correctness_of_work = 0;
	double a = 0;
	double b = 0;
	for (int i = 0; i < meaning.size(); i++) // підрахунок % правильності роботи
коду
	{
		if (new_messages[i].isSpam == true && meaning[i] == " SPAM")
		{
			a++;
			b++;
		}
		if (new_messages[i].isSpam == false && meaning[i] == " NOT SPAM")
		{
			a++;
			b++;
		}
		if (new_messages[i].isSpam == true && meaning[i] != " SPAM")
		{
			b++;
		}
		if (new_messages[i].isSpam == false && meaning[i] != " NOT SPAM")
		{
			b++;
		}

	}
	correctness_of_work = a / b;
	cout <<"Correctness of work : "<< correctness_of_work <<" % " << endl;

	return 0;
}
```